

PatchPilot: A Tool-Using Agent for Python Debugging and Repair

Team Members

Tivdzua Lubem Noah; Gisele Wiykiynyuy

Course

Agentic AI

A bounded Python repair agent that uses restricted tools, auditable traces, and executable tests to verify source-code fixes.

System components: local Qwen model backend, controlled repository tools, disposable workspaces, patch rollback, pytest verification, reproducible benchmark, Streamlit demonstration, Docker packaging, and hosted interface.

Abstract

PatchPilot is a bounded, tool-using system for Python debugging and repair. It reproduces failing tests, inspects relevant source files, applies controlled source patches, reruns the full pytest suite, and accepts a repair only when executable verification passes. The implementation combines a structured repair loop, a local Qwen2.5-Coder model through Ollama, a restricted repository tool boundary, disposable benchmark workspaces, patch rollback, execution budgets, and auditable traces.

The evaluation uses PatchPilot-Bench v0, a controlled mutation-seeded benchmark of 12 Python repair tasks across arithmetic, geometry, list processing, number handling, statistics, and string manipulation. The full agent repaired 12 of 12 tasks. A one-shot baseline repaired 8 of 12 tasks, and a no-retry reduced-budget ablation also repaired 8 of 12 tasks. The paired comparison shows that four tasks were repaired only by the full agent, supporting the value of iterative, verification-driven repair over single-pass patch generation.

Keywords: software repair; Python debugging; tool use; pytest verification; bounded agents; local language models.

Introduction

Software repair is a sequential activity. A developer observes a failure, identifies the relevant implementation, forms a hypothesis, applies a change, and verifies whether the new source revision is correct. PatchPilot operationalises this workflow as a bounded repair system with explicit tool use and verification rather than direct, unconstrained code editing.

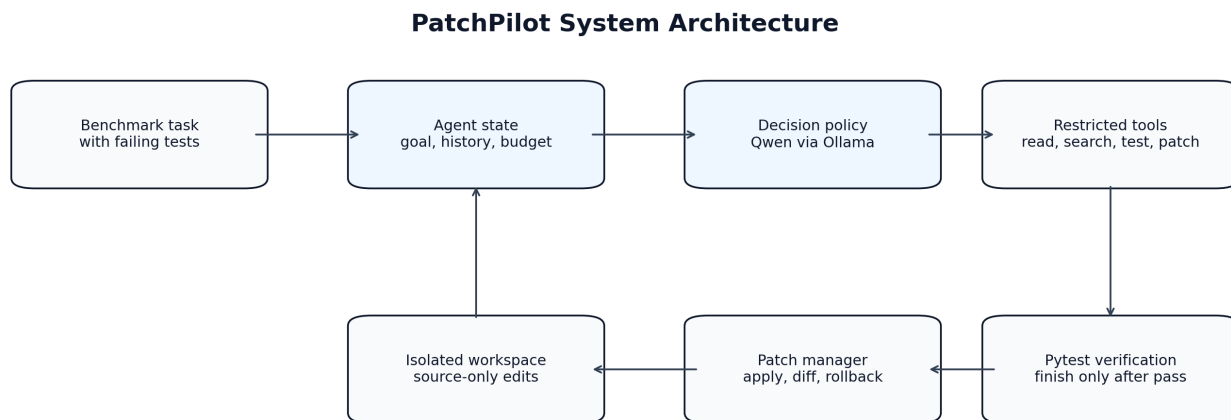
The project focuses on a practical question: can a small local model become a reliable repair component when its actions are constrained by repository tools, state tracking, retry budgets, and executable tests? PatchPilot is designed around that question. The model does not directly edit the repository; it proposes actions and patches that must pass validation before they affect the workspace.

Project Scope and Deliverables

The completed project includes a working repair agent, a 12-task Python benchmark, a one-shot baseline, a no-retry ablation, evaluation metrics, statistical analysis, a Dockerized Streamlit interface, a hosted demonstration page, and generated report figures and tables. The benchmark is described as controlled and mutation-seeded, with external large-scale validation left as future work.

System Design

PatchPilot separates repair decisions from repository authority. The decision policy receives compact state and observation summaries, then selects a structured action. The executor validates that action and dispatches it through a restricted tool boundary. This keeps the workspace controlled and makes the repair process auditable.



Every action passes through a validated tool boundary; tests are the final authority for success.

Figure 1. PatchPilot system architecture.

The core state records the repair goal, action history, observations, changed files, current verification status, budget usage, and patch attempts. Tools are limited to safe repository operations such as reading files, searching code, running tests, applying patches, viewing diffs, and restoring files. Tests are the final authority: the agent can finish successfully only after

the current source revision passes pytest.

Repair Workflow

The repair workflow follows a Plan-Act-Observe-Reflect-Verify structure. The system first reproduces the failure, then uses the observed test output to locate source code, inspect the implementation, generate a patch, apply it, and rerun the tests. If verification fails and budget remains, the agent reads the new state and attempts another bounded repair.

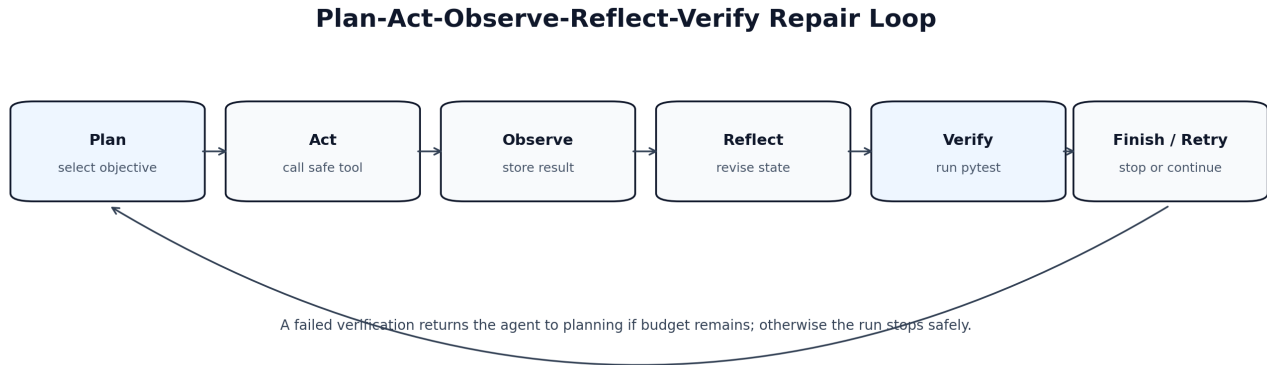


Figure 2. Plan-Act-Observe-Reflect-Verify repair loop.

Safety Controls

The tool boundary enforces source-only edits, rejects forbidden paths, limits tool calls and patch attempts, records traces, and supports rollback. Each benchmark run executes in a disposable workspace so that failed patches do not corrupt the source benchmark.

- Path restrictions prevent edits to regression tests and non-approved locations.
- Execution budgets bound the number of steps, tool calls, patch attempts, and elapsed time.
- Patch application is mediated by a patch manager that can record diffs and restore files.
- A run is reported as successful only after full-suite pytest verification.

Implementation

The implementation is organised as a Python package with separate modules for schemas, agent control, tool execution, patch management, model access, benchmark running, evaluation metrics, and demonstration code. The live model backend uses Ollama with qwen2.5-coder:1.5b, selected because it is a CPU-friendly open model for local execution.

The Streamlit interface exposes the workflow without requiring users to inspect code directly. It lets a user choose a benchmark task, view the broken source and tests, run a local repair, inspect the agent trace, see the patch diff, and confirm final verification. Docker support packages the interface and project environment for local execution outside the development shell.

Benchmark

PatchPilot-Bench v0 contains 12 controlled Python repair tasks. Each task includes a small repository, source files, pytest tests, a manifest, allowed edit paths, forbidden test paths, difficulty metadata, and expected initial failures. The tasks cover common defect categories such as wrong binary operators, wrong formulas, list-processing mistakes, numeric edge cases, statistics, and string handling.

Task	Domain	Defect type	Difficulty
calculator-001	Arithmetic	wrong binary operator	Easy
calculator-002	Arithmetic	wrong binary operator	Easy
calculator-003	Arithmetic	wrong formula/branch	Easy
calculator-004	Arithmetic	wrong edge case	Easy
geometry-001	Geometry	wrong formula	Easy

Task	Domain	Defect type	Difficulty
geometry-002	Geometry	wrong formula	Medium
lists-001	Lists	wrong list operation	Easy
lists-002	Lists	wrong boundary handling	Medium
numbers-001	Numbers	wrong numeric condition	Easy
stats-001	Statistics	wrong aggregation formula	Medium
strings-001	Strings	wrong string operation	Easy
strings-002	Strings	wrong string edge case	Medium

Table 1. PatchPilot-Bench v0 task catalog.

Evaluation Methodology

All evaluated conditions ran on the same 12 tasks with the same model backend. The conditions differ in control flow and budget so that the comparison isolates the effect of iterative verification and retry capacity.

Condition	Description
Full agent	Complete workflow with verification-driven retry budget up to three patch attempts.
One-shot baseline	Reproduce failure, inspect once, apply one patch, verify once, then stop.
No-retry ablation	Reduced-budget condition with one patch attempt and tighter step/tool limits.

Table 2. Evaluation conditions.

The main metrics are repair rate, full-suite pass rate, invalid patch rate, patch attempts, tool calls, execution time, escalation, and budget exhaustion. Full-suite pass rate is the most important correctness metric because PatchPilot reports success only after pytest verifies the final source revision.

Results

The full agent repaired every benchmark task. The one-shot baseline repaired 8 of 12 tasks and escalated on 4 tasks after its single patch did not pass verification. The no-retry ablation also repaired 8 of 12 tasks, with 4 failures caused by budget exhaustion. No condition produced invalid patch runs.

Condition	Repairs	Repair rate	Full-suite pass	Invalid patches	Failure mode	Mean patch attempts
Full agent	12/12	100.0%	100.0%	0.0%	None	1.25
One-shot baseline	8/12	66.7%	66.7%	0.0%	4 escalations	1.0
No-retry ablation	8/12	66.7%	66.7%	0.0%	4 budget exhaustions	1.0

Table 3. Evaluation summary.

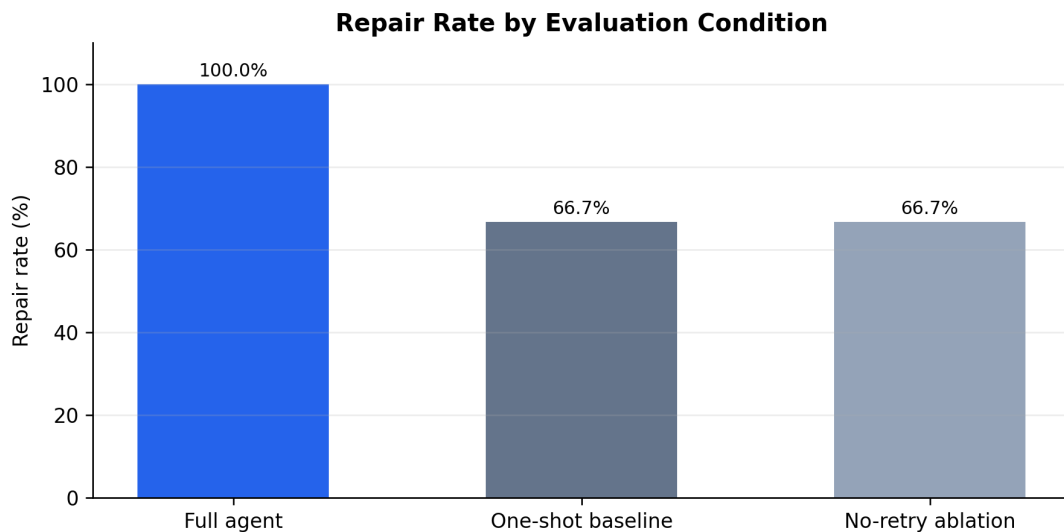


Figure 3. Repair rate comparison across evaluation conditions.

The result shows the practical effect of a verification-driven loop. The full agent solved four tasks that the one-shot baseline did not solve. Several full-agent runs used a second patch attempt, demonstrating that a failed first patch can still produce

useful evidence for a successful follow-up repair.

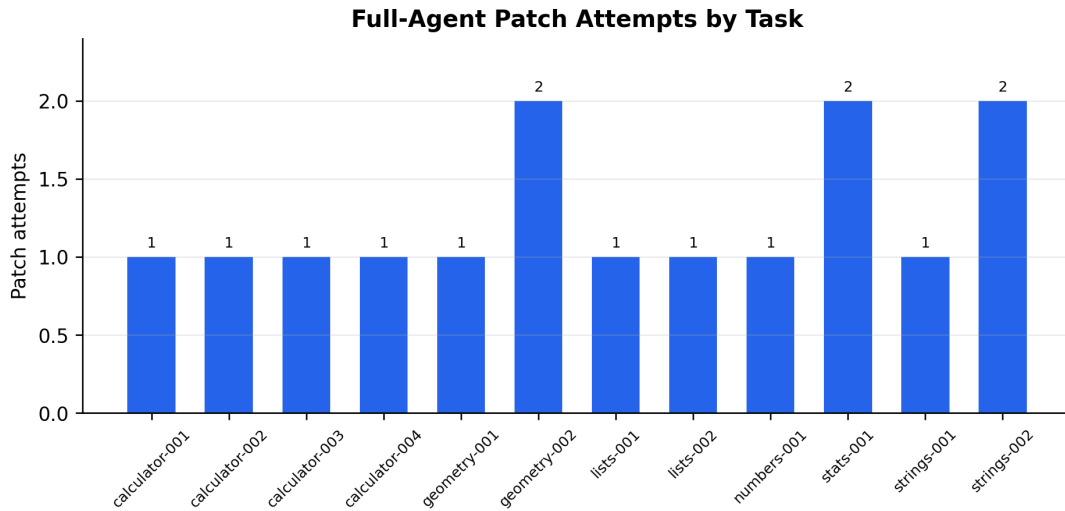


Figure 4. Patch attempts used by the full-agent condition.

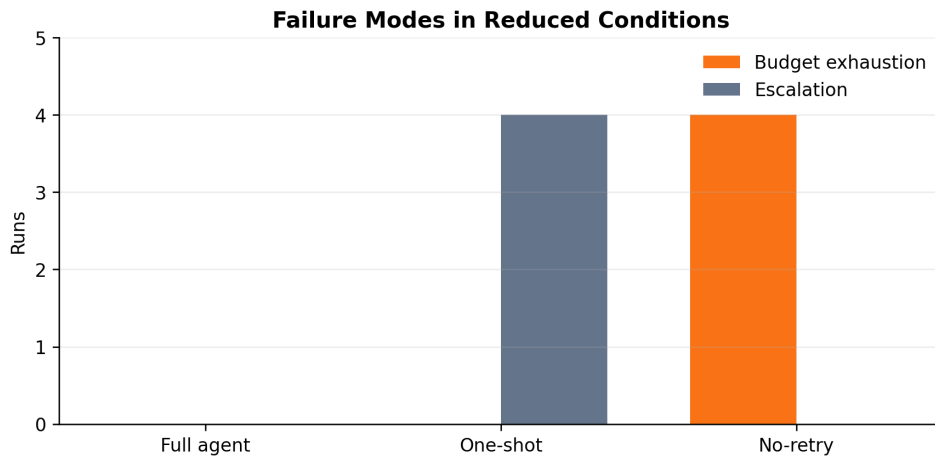


Figure 5. Failure modes in the reduced conditions.

Statistical Analysis

Because the full agent and one-shot baseline were evaluated on the same tasks, success can be compared as paired binary outcomes. The paired table contains 8 tasks solved by both systems, 4 tasks solved only by the full agent, 0 tasks solved only by the one-shot baseline, and 0 tasks failed by both systems.

Statistic	Value
Both succeeded	8
Full agent only succeeded	4
One-shot baseline only succeeded	0
Both failed	0
Discordant pairs	4
Exact McNemar two-sided p-value	0.1250

Table 4. Paired statistical analysis.

The McNemar p-value is 0.1250. With 12 paired tasks, this value should be interpreted cautiously. The clearest evidence is the effect size: four additional tasks were repaired only by the full agent, while no task was repaired only by the one-shot baseline.

Demonstration and Availability

The project includes both a local live demonstration and a hosted public interface. The local demonstration runs the actual repair pipeline with Ollama. The hosted page provides access to the interface and evaluation evidence, while live model execution remains local because the repair loop depends on an Ollama model server.

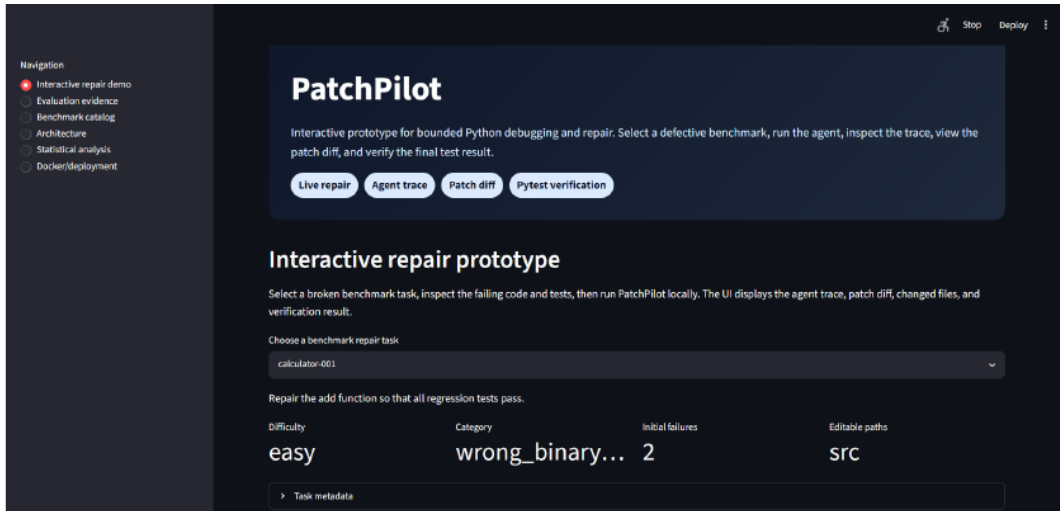


Figure 6. Local live repair demonstration showing successful pytest-verified repair.

Artefact	Location or role
GitHub repository	https://github.com/tivdzualubem/patchpilot-agentic-ai
Hosted interface	https://lubem-patchpilot-agentic-ai.hf.space/
Hugging Face Space	https://huggingface.co/spaces/lubem/patchpilot-agentic-ai
Live repair execution	Local Streamlit session connected to Ollama and qwen2.5-coder:1.5b.
Container support	Dockerfile and docker-compose configuration; local build and run verified.

Table 5. Project availability.

Limitations

The benchmark size is appropriate for the project scope but too small for broad claims about all Python repair problems. The tasks are controlled, small, and pure-Python; larger real-world repositories would introduce dependency management, environment reconstruction, and multi-file reasoning challenges beyond this version.

- The live evaluation uses one local model configuration rather than a broad model comparison.
- The hosted interface does not run live repairs unless connected to a model service.
- The paired statistical comparison has limited power because the benchmark contains 12 tasks.
- External datasets are left for future validation after dependency and environment automation are added.

Future Work

Future work should expand the benchmark, add multi-file and dependency-heavy tasks, compare additional model backends, include public repair datasets after environment support is mature, and provide hosted inference for live repairs in the public interface. Repeated runs with multiple seeds would also improve statistical confidence.

Conclusion

PatchPilot demonstrates a complete bounded repair workflow for Python debugging. It combines a local model, restricted tools, state tracking, patch control, rollback, executable verification, benchmark evaluation, Docker packaging, and a demonstration interface. The full agent repaired 12 of 12 benchmark tasks, while the one-shot baseline repaired 8 of 12. The four-task paired advantage supports the central design choice: iterative observation and verification can make a repair agent more reliable than a single-pass patch attempt.